# AUTHENTICATION SECURITY PROJECT CODE

GitHub-ready starter project • Node.js • JavaScript • Automated tests

## Project objective

Apply defensive authentication controls and demonstrate them with automated tests.

## Included controls

Password hashing with scrypt, server-side input validation, secure session cookies, logout, session expiry, rate limiting, generic login errors, and basic automated tests.

## Files included in this PDF

1. server.js
2. package.json
3. server.test.js
4. README.md
5. .gitignore

Educational starter project — not production-ready authentication.

# 1. server.js — Backend authentication

```
const http = require("node:http");
const crypto = require("node:crypto");

const users = new Map();
const sessions = new Map();
const attempts = new Map();

const SESSION_TIME = 15 * 60 * 1000;
const MAX_ATTEMPTS = 5;
const WINDOW = 10 * 60 * 1000;

function send(res, status, data, headers = {}) {
  res.writeHead(status, {
    "Content-Type": "application/json",
    "Cache-Control": "no-store",
    "X-Content-Type-Options": "nosniff",
    ...headers
  });
  res.end(JSON.stringify(data));
}

function readBody(req) {
  return new Promise((resolve, reject) => {
    let body = "";
    req.on("data", chunk => {
      body += chunk;
      if (body.length > 10000) {
        reject(new Error("Request too large"));
        req.destroy();
      }
    });
    req.on("end", () => {
      try { resolve(JSON.parse(body || "{}")); }
      catch { reject(new Error("Invalid JSON")); }
    });
    req.on("error", reject);
  });
}

function hashPassword(password, salt = crypto.randomBytes(16).toString("hex")) {
  return {
    salt,
    hash: crypto.scryptSync(password, salt, 64).toString("hex")
  };
}

function verifyPassword(password, record) {
  const actual = crypto.scryptSync(password, record.salt, 64);
  const expected = Buffer.from(record.hash, "hex");
  return actual.length === expected.length &&
    crypto.timingSafeEqual(actual, expected);
}

function validInput(username, password) {
  return typeof username === "string" &&
    /^[a-zA-Z0-9_.-]{3,32}$/.test(username) &&
    typeof password === "string" &&
    password.length >= 12 && password.length <= 128;
}

function getCookie(req) {
  const cookies = req.headers.cookie || "";
  const item = cookies.split(";").map(x => x.trim())
    .find(x => x.startsWith("sid="));
  return item ? decodeURIComponent(item.slice(4)) : null;
}

function rateLimited(req) {
  const key = req.socket.remoteAddress || "unknown";
  const now = Date.now();
  const recent = (attempts.get(key) || [])
    .filter(time => now - time < WINDOW);
  attempts.set(key, recent);
  return recent.length >= MAX_ATTEMPTS;
}

function recordAttempt(req) {
  const key = req.socket.remoteAddress || "unknown";
  const recent = attempts.get(key) || [];
  recent.push(Date.now());
  attempts.set(key, recent);
}

function makeCookie(token, maxAge) {
```

```
  const secure = process.env.NODE_ENV === "production" ? "; Secure" : "";
  return `sid=${token}; HttpOnly; SameSite=Strict; Path=/; Max-Age=${maxAge}${secure}`;
}

const server = http.createServer(async (req, res) => {
  const url = new URL(req.url, "http://localhost");

  if (req.method === "GET" && url.pathname === "/api/health") {
    return send(res, 200, { status: "running" });
  }

  if (req.method === "POST" && url.pathname === "/api/register") {
    let data;
    try { data = await readBody(req); }
    catch { return send(res, 400, { error: "Invalid request." }); }

    const { username, password } = data;
    if (!validInput(username, password)) {
      return send(res, 400, {
        error: "Use a valid username and a password of 12-128 characters."
      });
    }

    const key = username.toLowerCase();
    if (users.has(key)) {
      return send(res, 409, {
        error: "Unable to create account with those details."
      });
    }

    users.set(key, { username, ...hashPassword(password) });
    return send(res, 201, { message: "Account created." });
  }

  if (req.method === "POST" && url.pathname === "/api/login") {
    if (rateLimited(req)) {
      return send(res, 429, {
        error: "Too many attempts. Try again later."
      }, { "Retry-After": "600" });
    }

    let data;
    try { data = await readBody(req); }
    catch {
      recordAttempt(req);
      return send(res, 400, { error: "Invalid request." });
    }

    const username = typeof data.username === "string"
      ? data.username.toLowerCase() : "";
    const password = typeof data.password === "string" ? data.password : "";
    const user = users.get(username);
    recordAttempt(req);

    if (!user || !verifyPassword(password, user)) {
      return send(res, 401, {
        error: "Invalid username or password."
      });
    }

    const token = crypto.randomBytes(32).toString("hex");
    sessions.set(token, {
      username: user.username,
      expiresAt: Date.now() + SESSION_TIME
    });

    return send(res, 200, { message: "Login successful." }, {
      "Set-Cookie": makeCookie(token, 900)
    });
  }

  if (req.method === "GET" && url.pathname === "/api/me") {
    const token = getCookie(req);
    const session = token ? sessions.get(token) : null;
    if (!session || session.expiresAt <= Date.now()) {
      if (token) sessions.delete(token);
      return send(res, 401, { error: "Please sign in." });
    }
    return send(res, 200, { username: session.username });
  }

  if (req.method === "POST" && url.pathname === "/api/logout") {
    const token = getCookie(req);
    if (token) sessions.delete(token);
    return send(res, 200, { message: "Logged out." }, {
      "Set-Cookie": makeCookie("", 0)
```

```
    });
  }

  return send(res, 404, { error: "Not found." });
});

if (require.main === module) {
  const port = process.env.PORT || 3000;
  server.listen(port, () => {
    console.log(`Server running at http://localhost:${port}`);
  });
}

module.exports = { server, hashPassword, verifyPassword, validInput };
```

## 2. package.json — Project scripts

```
{
  "name": "authentication-security-demo",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "start": "node server.js",
    "test": "node --test"
  },
  "engines": {
    "node": ">=20"
  }
}
```

## 3. server.test.js — Automated tests

```
const test = require("node:test");
const assert = require("node:assert/strict");

const {
  hashPassword,
  verifyPassword,
  validInput
} = require("./server");

test("password is stored as a hash", () => {
  const password = "my-long-test-password";
  const record = hashPassword(password);

  assert.notEqual(record.hash, password);
  assert.equal(verifyPassword(password, record), true);
  assert.equal(verifyPassword("wrong-password", record), false);
});

test("validates username and password", () => {
  assert.equal(validInput("student01", "long-test-password"), true);
  assert.equal(validInput("x", "short"), false);
  assert.equal(validInput("bad name", "long-test-password"), false);
});
```

## 4. README.md — Documentation

```
# Authentication Security Demo

## Objective
Apply defensive authentication controls and demonstrate them with automated tests.

## Features
- Password hashing with Node.js scrypt
- Server-side input validation
- HttpOnly, SameSite session cookies
- Logout and 15-minute session expiry
- Login rate limiting
- Generic authentication error messages
- Automated tests

## Requirements
Node.js 20 or later.

## Run
```bash
npm start
npm test
```

Check the health endpoint at http://localhost:3000/api/health.

## Security note
This is an educational starter project. Users, sessions, and rate-limit counters are held in memory and reset on restart
. Production use requires persistent storage, CSRF protection and Origin checks for cookie-authenticated actions, HTTPS,
 shared rate limiting, monitoring, and a security review. Never commit credentials, tokens, or sensitive user data.
```

## 5. .gitignore — Files excluded from Git

```
node_modules/
.env
cookies.txt
.DS_Store
```

## How to use the project

• Create a folder named authentication-security-demo.

• Create the five files shown in this PDF and paste each code section into its matching file.

• Install Node.js 20 or later.

• Open a terminal in the folder and run: npm start

• Open a second terminal and run: npm test

• Upload the files to a GitHub repository and include test output as proof.

**Deployment note:** This backend needs a Node.js-compatible host; GitHub Pages alone only serves static files.